

Hermes Agent 架构解析

<https://zhuanlan.zhihu.com/p/2027690167671039927>

Hermes 是 [Nous Research](#) 开源的 Agent 框架，GitHub 上 Stars 破了 8 万，多家大厂内部也在研究这个架构。它的核心理念只有一句话：与你共同成长。不是「帮你完成任务」，是「和你一起变强」。

这篇文章，我想把 Hermes 的架构从头拆一遍。不是翻译文档，是把我觉得真正值得学的地方说清楚。



一、先说一个根本问题

为什么大多数 Agent 用着用着就觉得「不够用」？

原因其实很简单——它们本质上是无状态的。每次对话重新开始，之前聊过什么、用过哪些工具、犯过什么错，一概不记得。就像你每天上班，同事都忘了你昨天说过的话，还得重新介绍一遍自己。

这带来了三个实际问题：

- 第一，没有积累。你教它怎么操作你们公司的内部系统，下次还得再教一遍。每次对话都在消耗同样的 Token，做同样的事情。
- 第二，没有自检。它不知道自己哪次做对了、哪次做错了，更不会主动改进。出了问题只能靠你发现，靠你纠正。
- 第三，上下文管理混乱。一旦对话稍微长一点，它开始「忘事」。要么把所有历史塞进 Context 导致 Token 爆炸，要么截断历史导致前后矛盾。

这三个问题，Hermes 的设计目标就是一一解决它们。

二、整体架构：三层骨架，六个系统

我习惯把 Hermes 的架构理解为三层结构，每一层对应一种能力。



- 第一层：感知层（Perception）
 - 负责「看懂世界」。用户发来的消息、当前对话的上下文、外部 API 返回的数据，都在这一层被消化。它做的事情是：这个输入是什么意思？用户真正想要的是什么？当前状态是什么？
- 第二层：决策层（Cognition）
 - 负责「想清楚该做什么」。任务拆解在这里发生，规划在这里生成，记忆在这里被调取。这里是整个 Agent 的「大脑」，也是 Hermes 最花心思的地方。
- 第三层：执行层（Action）

- 负责「真正把事做完」。调用工具、操作文件、访问 API、执行代码——这些都在执行层完成。大多数 Agent 框架把这层做得很薄，但 Hermes 在这里加了相当重的工程保障。

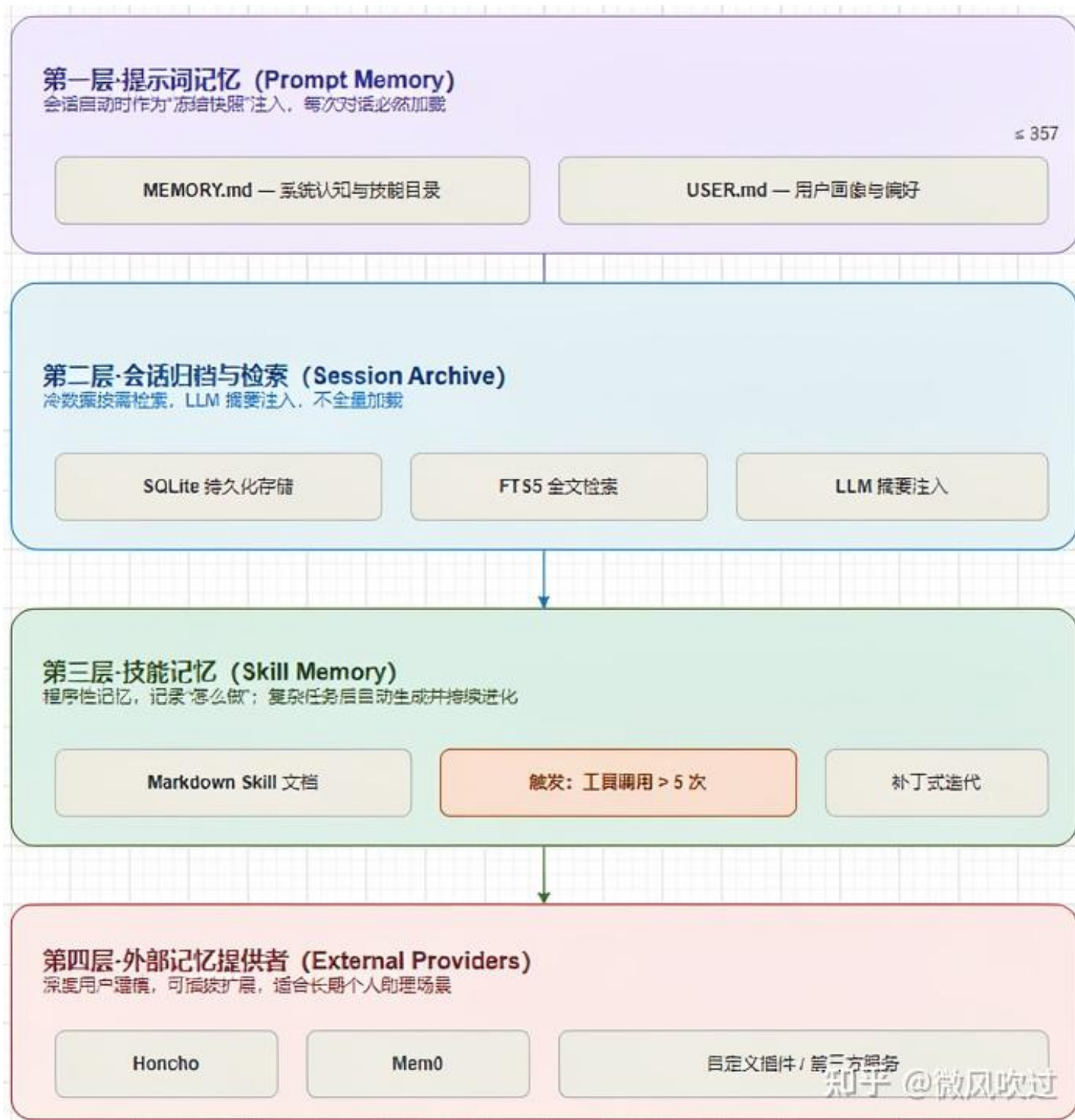
三层之上，Hermes 定义了六个子系统，把整个运行链路串起来：

1. **用户消息触发器**：接收输入，同时触发技能创建流程和轮次计数。
2. **周期性 Nudge（提示机制）**：这是 Hermes 的一个核心创新——每执行到一定轮次，系统会自动产生一个「该复盘了」的内部信号，把学习从用户要主动触发变成 Agent 自己的本能。
3. **后台复盘（Background Review）**：独立的守护进程，异步运行，专门做总结和分析，完全不阻塞主对话流程。
4. **双文件存储**：本地文件加云端备份，确保记忆能真正持久化。
5. **全息记忆（Holographic Memory）**：长期的、结构化的知识存储。
6. **记忆管理器（Memory Manager）**：短期记忆的调度中枢。

这六个系统的分工，是 Hermes 和大多数 Agent 框架最大的不同——它是一套完整的、有分工的操作系统，而不是一个被 Prompt 驱动脚本。

三、记忆系统：四层设计，各司其职

如果说架构是骨骼，记忆系统就是 Hermes 的血液。



很多 Agent 框架说自己有「记忆」，实际上不过是把聊天历史扔进 Context，或者用个向量数据库做召回。这种方式有个致命问题：随着使用时间增长，它反而会越来越乱。什么都往里存，什么都要检索，结果信噪比越来越低。

Hermes 的记忆系统是分层的，每一层解决不同的问题。

- 第一层：提示词记忆 (Prompt Memory)

这是最核心的一层，对应两个文件：

- MEMORY.md：存储系统对自身的认知，比如它掌握了哪些技能、有哪些操作规范。
- USER.md：存储用户画像，比如用户的习惯、偏好、常用的工作流。

每次对话启动时，这两个文件会作为「冻结快照」注入到 Context 里。容量被严格限制在约 3575 字符——不是技术限制，是主动控制。这层的逻辑是：把最重要的东西固定住，保证每次对话都有一个稳定的「底座」。

- 第二层：会话归档与检索（Session Archive）

这一层处理「冷数据」。

历史对话被存进 SQLite，并启用了 FTS5 全文检索。当前对话需要引用历史信息时（比如「上周我们讨论过的那个方案」），系统按需检索，用 LLM 做摘要，把精炼后的内容注入上下文。

这个设计的精妙之处在于：不是把历史对话全部召回，而是只取你现在需要的那一部分，并且以摘要形式注入，而不是原文复现。Token 消耗大幅降低，同时信息密度提高了。

- 第三层：技能记忆（Skill Memory）

这是程序性记忆，记录的是「怎么做」，而不是「发生了什么」。

当一次任务复杂度超过一定阈值（比如工具调用超过 5 次），系统会自动把这次任务的执行过程提炼成一个 Skill 文档，遵循 <http://agentskills.io> 的标准格式，保存为 Markdown 文件。

下次遇到类似任务，Agent 可以直接调用这个 Skill，而不是重新摸索。随着使用积累，这些 Skill 还会通过「补丁」方式迭代更新——用得越多，越精准。

这才是「与你共同成长」这句话真正落地的地方。

- 第四层：外部记忆提供者

这是扩展接口，支持接入 Honcho、Mem0 等外部服务，做更深度的用户建模。对于需要长期追踪用户偏好、行为模式的场景，这一层提供了扩展空间。

四层加起来，Hermes 的记忆系统是有梯度的：高频核心信息放顶层，历史数据按需检索，技能经验结构化沉淀，深度建模留给专业工具。不是什么都往一个地方塞，而是各司其职。

四、技能与规划系统：闭合学习回路

学会了怎么记忆，接下来说 Hermes 怎么「变聪明」。

闭合学习回路

Hermes 的技能系统形成了一个完整的闭合回路：

执行 → 评估 → 提炼 → 进化

每执行 15 次工具调用，系统会自动做一次自检，评估这段时间的操作是否达到预期效果。如果有值得提炼的经验，就生成或更新对应的 Skill 文档。如果之前的 Skill 有不准确的地方，会以「补丁」形式修正，而不是覆盖重写。

这个设计避免了两个常见的坑：

- 一是「学了就忘」，每次任务的经验都能沉淀下来；

- 二是「一学定终身」，Skill 会持续迭代，不会因为早期经验不准确就永远跑偏。

渐进式披露

随着使用时间增长，Skill 会越来越多，如果每次都全部加载，Token 消耗会指数级增长。**Hermes 的解法是渐进式披露：默认只加载 Skill 的摘要版本，只有在真正需要执行某个 Skill 的时候，才加载完整的步骤。**

这是一个很实用的工程细节，真正在生产环境里跑过 Agent 的人会懂这个设计有多重要。

规划引擎（Planner）

很多 Agent 的「规划」其实是假的——把任务描述扔给模型，让它生成一个步骤列表，然后按顺序执行。这种方式在任务稍微复杂一点、中途出现意外的情况下，成功率非常低。

Hermes 的 Planner 支持 Plan → Execute → Replan 的动态循环。不是一次性输出计划就完事，而是执行过程中持续观察结果，一旦发现偏差就重新规划。这才像一个真正「做项目」的人：不是只会写计划，而是会根据实际情况调整计划。

五、工程化：把每个细节都想清楚

架构设计好不代表真的能用。我觉得 Hermes 真正有价值的，是它在工程层面的认真程度。

异步复盘，不打扰用户

Background Review 是一个独立的进程，通过 Fork 机制运行，和主对话流程完全解耦。

这意味着：你在和 Agent 对话的时候，它在后台悄悄做总结、提炼技能——但你不会感受到任何延迟。这不是小细节，这是一个系统能不能真正好用的关键区别。很多 Agent 的「反思」和「总结」是同步执行的，每次都会有一段等待时间，用户体验极差。

多平台网关

一套核心代码，通过 Gateway 进程接入 Telegram、Discord、飞书、钉钉等 15 个以上的平台。

对于企业落地来说，这个设计价值很大。员工用什么 IM 工具，Agent 就接在哪里，不需要为每个平台单独维护一套逻辑。

灵活的部署方式

支持 Local、Docker、SSH、Modal 等多种后端环境，从 5 美元一个月的 VPS 到 GPU 集群都能跑。这种设计让 Hermes 的使用门槛很低，同时也支持规模化扩展。

五层安全机制

自进化这件事最让人担心的是失控：Agent 自己改自己，边界在哪？

Hermes 设计了五层防线：用户授权、操作审批、容器隔离、凭证过滤、注入扫描。每一层针对不同的风险点。可观测性也是显式设计的——任务执行路径可以被监控和审计，出了问题能追溯。

六、和主流框架比，Hermes 的位置在哪？

Hermes 最大的优势在长期使用场景——需要持续积累经验、需要跨会话保持记忆、需要随着使用变得越来越好用的场景。

它的局限也是真实的：复杂任务的执行路径对用户来说仍然是黑盒，在需要完全可解释、每步都要审计的企业合规场景下，目前还需要做不少定制。

七、如果你要落地，三个建议

1. 第一，从单一任务切入，不要一上来做大而全的 Agent。

Hermes 的架构支持「从小做大」。先把一个具体的、高频的任务场景跑通，让 Skill 系统积累起来，再逐步扩展。想一下子覆盖所有场景，只会让调试变成噩梦。

2. 第二，优先建设执行能力，而不是对话能力。

能帮你真正把事做完，比能和你聊得天花乱坠重要得多。工具调用稳不稳、任务中断能不能恢复、多步骤执行的成功率高不高——这些才是一个 Agent 值不值得用的核心指标。

3. 第三，把业务能力 Skill 化。

把你们公司高频的操作流程整理成 Skill 格式，这件事本身就有价值——它会逼着你把之前模糊的、全靠人脑记忆的操作规范显式化。这些 Skill 沉淀下来，是真正可复用、可迭代的资产。

最后说一句

我觉得 Hermes 代表的不只是一个具体框架，而是一种对 Agent 的不同理解：

大模型是大脑，但光有大脑不够。真正能用的 Agent，需要的是一套完整的神经系统——能感知、能规划、能记忆、能执行、能学习、能自我修正。

AI Agent 的竞争，归根结底是工程能力的竞争。状态怎么管、记忆怎么分层、异步怎么处理、失败怎么恢复——这些「枯燥」的工程问题，才是决定上限的地方。